ACADEMIC TASK-1

CSE316

(Operating System)

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Name	Registration No.	Roll No.
Prerna Shukla	12409006	23
Kuppala Naga Sai Vinay	12413452	24
Dev Kumar	12409246	22

Section: 424GB

Date: 24/04/2026

LOVELY PROFESSIONAL UNIVERSITY

Title : Project Report: Deadlock Detection and Recovery Toolkit

1. Project Overview

A deadlock occurs when two or more processes are stuck waiting for resources held by each other, causing the system to freeze and halt execution. Traditional deadlock detection methods rely on static algorithms such as **Wait-For Graphs, Resource Allocation Matrices, and Banker's Algorithm**, but these approaches struggle in **dynamic and large-scale systems**, especially cloud and distributed environments where resource usage changes rapidly.

The Deadlock Detection and Recovery Toolkit System **aims to intelligently monitor process behavior and resource allocation using machine learning models. Instead of detecting deadlocks after they occur, this system focuses on** predictive detection**, where patterns, anomalies, and historical data are analyzed to identify potential deadlocks *before* they impact performance.**

Goals of the Project

- **Detect deadlocks in real-time with higher accuracy compared to traditional algorithms.**
- **Predict potential deadlocks using machine learning models (e.g., classification, anomaly detection).**
- **Improve overall system reliability and reduce performance degradation due to blocked processes.**
- **Provide visualization tools and alerts to system administrators for better decision-making.**
- **Integrate traditional OS-level monitoring**

Project Scope

This project focuses on building an intelligent system that monitors system processes, analyzes resource usage data, and uses machine learning to detect and predict deadlocks in real time. It involves collecting system logs, converting them into useful features, and training models to identify patterns that lead to deadlocks. The system will be integrated with a simulated environment and will provide warnings through a dashboard interface. Finally, the project compares the approach with traditional deadlock detection methods to show improvements in accuracy and performance.

Expected Outcomes

- **A working system that detects deadlocks and predicts them in advance.**
- **A model capable of identifying resource patterns leading to circular waits.**
- **Real-time dashboard showing system processes, resource allocation, and alerts.**
- **Improved throughput and reduced system downtime.**
- **Experimental results demonstrating performance improvement over traditional approaches.**

2. Module-Wise Breakdown

Module 1: System Monitoring & Data Collection

Purpose:
To continuously track system processes and resource usage that may lead to deadlocks.

Role in the Project:

- **Monitors running processes, resource allocation, lock requests, and wait states.**
- **Collects real-time logs and historical data related to resource utilization.**
- **Sends structured data to the Machine Learning module for analysis.**
  This module acts as the data source for prediction and detection.

Module 2: Deadlock Detection and Recovery Toolkit

Purpose:
To analyze system data and use machine learning to identify or predict deadlocks.

Role in the Project:

- **Preprocesses and converts raw logs into meaningful features.**
- **Applies ML models (e.g., Decision Trees, Random Forest, Neural Networks) to detect patterns.**
- **Predicts potential deadlock scenarios and identifies ongoing deadlocks.**
- **Outputs status codes such as *Normal*, *Warning*, or *Deadlock Detected*.**
  This module forms the intelligence layer of the system.

Module 3: Visualization & User Interface

Purpose:
To display system status, alerts, and patterns in a clear visual format.

Role in the Project:

- **Provides a dashboard showing processes, resource flow, and alerts.**
- **Visualizes wait-for graphs, resource usage trends, and prediction results.**
- **Allows users to view logs, track history, and take corrective actions if implemented.**
  This module improves usability and helps administrators interpret results easily.

3. Functionalities

Module 1: System Monitoring & Data Collection

- Process tracking
- Resource allocation tracking
- Event logging
- Data preprocessing
- Dataset generatio

Module 2: AI-Based Detection & Prediction

- Feature extraction
- Model training
- Deadlock detection
- Deadlock prediction
- Risk classification

Module 3: Visualization & UI

- Dashboard display
- Graph visualization
- Alerts & notifications
- Log history view
- Manual control actions (optional)

4. Technology Used

- Programming Languages:
- Python
- HTML, CSS, JavaScript *(for UI)*

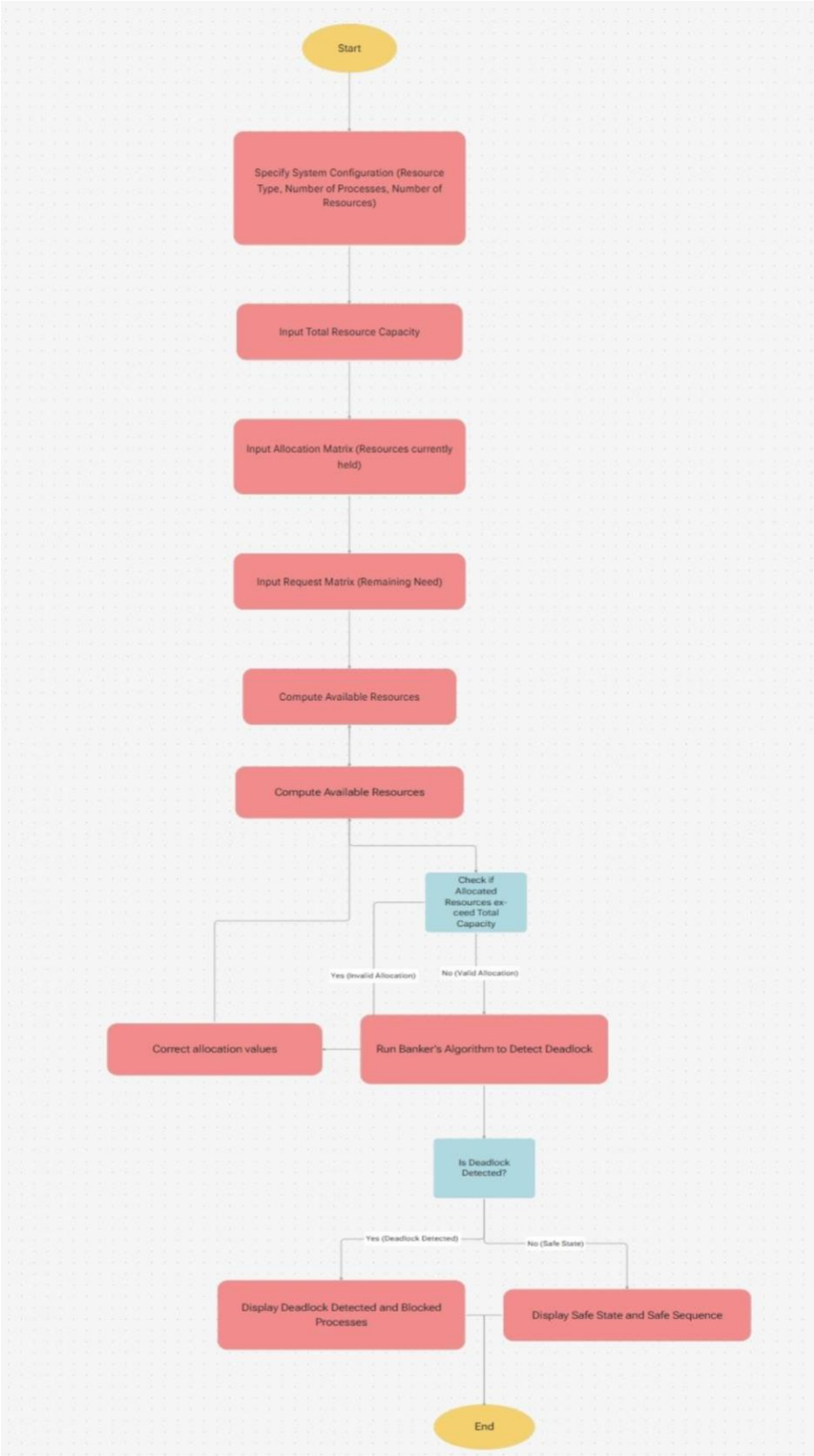
- **<u>Libraries and Tools:</u>**

- **psutil → process and resource monitoring**
- **pandas, numpy → data handling and preprocessing**
- **scikit-learn → machine learning models**
- **matplotlib / Plotly → visualization**
- **Flask / Django → backend and API for dashboard**

- **<u>Other Tools</u>:**

- **GitHub → version control**
- **VS Code → development environment**
- **Jupyter Notebook → ML model testing**
- **MySQL / SQLite → data storage (optional)**

5. System Flow Diagram



6. GitHub Tracking

Repository: Deadlock-Detection-and-Recovery-Toolkit

Link: : https://github.com/Vinay36/Deadlock-Detection-and-Recovery-Toolkit.git

Youtube link:https://youtu.be /X8Fx8hp2ynh1?feature=shared

---

7. Conclusion & Future Scope

Conclusion :

The Deadlock-Detection-and-Recovery-Toolkit System helps detect and predict deadlocks more efficiently than traditional methods. By using machine learning and real-time monitoring, it improves system performance, reduces process blocking, and enhances resource management.

Future Scope :

- **Can be integrated with real operating systems.**
- **Can use advanced AI models for higher accuracy.**
- **Can be expanded to cloud and distributed systems.**
- **Can include automatic deadlock recovery solutions.**

8. References

1. **Programming Languages**

   Python → Core implementation

   HTML, CSS (via Streamlit Components) → UI rendering inside Streamlit

2. **Libraries and Tools Used**

   Deadlock Simulation & OS Logic

NumPy → Matrix operations for Banker's Algorithm

Pandas → Table data handling for Allocation/Request matrices

Machine Learning (AI Prediction Model)

3. Backend Logic

Built completely inside Streamlit's session_state for:

Transferring simulation data → AI page

Storing trained model

Updating comparison results

4. Deployment / Development Tools

GitHub → Version Control & Team Collaboration

VS Code → Primary Code Editor

Appendix

A . Project Summary

- Focuses on detecting and predicting deadlocks using machine learning.
- Monitors processes, resources, and system states in real time.
- Converts system logs into datasets for analysis and ML model training.
- Improves system efficiency by identifying risks before deadlocks occur.

Module Breakdown

1. System Monitoring & Data Collection

- Tracks active processes and execution states
- Monitors resource allocation and waiting conditions
- Logs system events (requests, allocation, blocking)
- Generates structured datasets for ML processing

2. AI/ML-Based Detection & Prediction

- Preprocesses collected data and extracts features
- Uses ML models to detect ongoing deadlocks

- **Predicts future deadlock risks based on patterns**
- **Classifies system status: *Normal / Warning / Deadlock***

3. <u>Visualization & User Interface</u>

- **Displays system status on dashboard**
- **Provides graphical wait-for diagrams**
- **Shows alerts and notifications for detected risks**
- **Maintains history/logs of past deadlocks**

Key Functionalities

- **Process tracking**
- **Resource monitoring**
- **Dataset generation**
- **Feature extraction**
- **Machine learning classification**
- **Visual analytics and alerts**
- **Historical log storage**

Technology Stack

- **Languages: Python, HTML, CSS, JavaScript**
- **Libraries: psutil, pandas, numpy, scikit-learn, matplotlib/Plotly, Flask**
- **Tools: VS Code, GitHub, Jupyter Notebook, SQLite/MySQL**

● Outcome

- **Detects and predicts deadlocks proactively**
- **Reduces system delays and resource conflicts**
- **Enhances performance in dynamic or distributed environments**

B.Problem Statement

Deadlocks occur in computing environments when multiple processes compete for system

resources in such a way that each process waits indefinitely for resources held by others, resulting in complete system halt or performance degradation. Traditional techniques for detecting deadlocks, such as resource allocation graphs and wait-for graphs, are often limited to static environments and fail to respond effectively in dynamic, large-scale, or distributed systems where resource usage patterns frequently change.

There is a need for an intelligent system that not only detects existing deadlocks but can also predict potential deadlock conditions before they occur. This project aims to develop an AI-powered deadlock detection system capable of monitoring real-time process and resource data, applying machine learning techniques to analyze patterns, and providing alerts or warnings to prevent system failure.

AI Response:

1. Project Overview

The Deadlock-Detection-and-Recovery-Toolkit **aims to identify and predict deadlocks by monitoring system processes and resource dependencies using machine learning techniques. Instead of detecting deadlocks only after they occur, the system analyzes patterns in real-time data to provide early warnings and prevent system halts.**

Goals

- **To detect deadlocks accurately using machine learning models.**
- **To predict potential deadlocks before they occur.**
- **To improve system reliability and resource utilization.**
- **To provide real-time alerts and data visualization for monitoring.**

Expected Outcomes

- **A functioning system that monitors processes and identifies deadlock conditions.**
- **Predictive analytics that highlight high-risk deadlock scenarios.**
- **A dashboard displaying system status, alerts, and resource usage patterns.**
- **Reduced system delays and improved performance compared to traditional methods.**

Scope

- **Collecting process and resource data from system activity.**

- **Converting raw logs into structured datasets for ML analysis.**
- **Implementing ML-based detection and classification models.**
- **Visualizing system behavior through a user-friendly interface.**
- **Evaluating results against conventional deadlock detection techniques.**

---

2. Module-Wise Breakdown

Module 1: System Monitoring & Data Collection

Purpose:
To continuously track processes, resource allocation, and system activity required for deadlock analysis.

Role in the Project:

- **Collects real-time data on running processes and resource usage**
- **Logs events like resource requests and blocked states**
- **Provides structured input data to the AI module for analysis**

Module 2: Deadlock Detection and

Recovery Toolkit Purpose:
To analyze collected data using machine learning techniques and detect or predict deadlock scenarios.

Role in the Project:

- **Processes and extracts meaningful features from system data**
- **Performs classification to detect ongoing deadlocks**
- **Predicts future deadlock risks and flags system status as Normal, Warning, or Deadlock**

Module 3: Visualization & User Interface

Purpose:

To visually present system status, alerts, and deadlock information to the user in an understandable format.

Role in the Project:

- **Displays system status, graphs, and deadlock alerts on a dashboard**
- **Visualizes wait-for graphs and resource dependencies**
- **Shows history logs and allows user interpretation and manual action if required**

---

3. Functionalities

Module 1: System Monitoring & Data Collection – Functionalities

- **Process tracking**
- **Resource allocation monitoring**
- **Event logging**
- **Wait-state detection**
- **Dataset generation for ML**

Module 2: Deadlock Detection and Recovery Toolkit – Functionalities

- **Data preprocessing**
- **Feature extraction**
- **Machine learning model training**
- **Deadlock detection (current state)**
- **Deadlock prediction (future risk)**
- **System status classification**

Module 3: Visualization & User Interface – Functionalities

- **Dashboard display**

- **Graph visualization (e.g., wait-for graph)**
- **Alerts and notifications**
- **Historical log display**
- **User interaction/control options (optional)**

---

4. Technology Recommendations

Programming Languages

- **Python *(primary for backend, ML, data processing)***
- **HTML, CSS, JavaScript *(for front-end interface)***

Libraries & Frameworks

- **psutil → system/process monitoring**
- **pandas, numpy → data handling and preprocessing**
- **scikit-learn → machine learning models**
- **matplotlib / Plotly → data visualization**
- **Flask / Django → backend web framework for UI integration**

Databases

- **SQLite *(lightweight and easy to integrate)***
- **MySQL *(for large-scale system data storage)***

Development Tools

- **VS Code / PyCharm *(development environment)***
- **GitHub *(version control and collaborative development)***

---

5. Execution Plan

The execution of the *Deadlock Detection and Recovery Toolkit System* will be carried out in multiple phases to ensure systematic development, testing, and deployment.

Phase 1: Requirement Analysis & Planning

- Understand system behavior and deadlock conditions
- Identify required data sources, resources, and environment setup

Phase 2: Data Collection & Monitoring Setup

- Configure process and resource monitoring tools
- Generate and store system logs required for model training

Phase 3: Data Preprocessing & Feature Engineering

- Clean and convert raw logs into structured datasets
- Extract meaningful attributes related to resource allocation and wait cycles

Phase 4: Model Development & Training

- Select suitable ML techniques (e.g., Decision Trees, Random Forest)
- Train and validate model using labeled datasets

Phase 5: System Integration

- Connect monitoring module with ML prediction engine
- Implement API or backend services to process real-time data

Phase 6: UI & Visualization Development

- Create dashboard to display system status, graphs, and alerts
- Implement visual wait-for graphs and event logs

Phase 7: Testing & Evaluation

- Test system using real or simulated deadlock conditions
- Compare AI detection accuracy with traditional techniques

Phase 8: Deployment & Documentation

- Deploy the system in a test environment
- Prepare final documentation, report, and project presentation

B . Solution Implementation :

LOVELY PROFESSIONAL UNIVERSITY

```python
import streamlit as st
import pandas as pd
import numpy as np
import time


st.set_page_config(
    page_title="Deadlock Simulation",
    page_icon="🛡️",
    layout="wide",
    initial_sidebar_state="collapsed"
)


st.markdown("""
<style>
    @import url('https://fonts.googleapis.com/css2?family=Inter:wght@400;600;800&display=swap');
    html, body, [class*="css"] { font-family: 'Inter', sans-serif; }

    .result-box-safe {
        background-color: #ecfdf5;
        border: 2px solid #10b981;
        padding: 20px;
        border-radius: 10px;
        color: #064e3b;
        text-align: center;
        margin-top: 20px;
    }
    .result-box-dead {
        background-color: #fef2f2;
        border: 2px solid #ef4444;
        padding: 20px;
        border-radius: 10px;
        color: #7f1d1d;
        text-align: center;
        margin-top: 20px;
    }
    div.stButton > button {
        background: linear-gradient(90deg, #2563eb 0%, #4f46e5 100%);
        color: white;
        border: none;
        padding: 0.75rem 1rem;
        font-weight: 600;
        border-radius: 0.5rem;
        width: 100%;
        transition: all 0.2s ease;
        box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    }
    div.stButton > button:hover {
        transform: scale(1.02);
        box-shadow: 0 6px 10px rgba(37, 99, 235, 0.2);
    }
```

LOVELY PROFESSIONAL UNIVERSITY

```
</style>
""", unsafe_allow_html=True)


st.title("🛡️ Deadlock Simulation")

st.markdown("""
This page simulates **Banker's Algorithm** for:
- ✅ Single-instance resources
- ✅ Multiple-instance resources

You first specify **total capacity** of each resource, then how many
resources are **allocated** to each process and what each process is
currently **requesting (remaining need)**.
The app automatically computes **Available** and checks for **deadlock**.
""")

st.markdown("---")


with st.container(border=True):
    st.subheader("⚙️ System Configuration")

    col_mode, col_p, col_r = st.columns([1.5, 1, 1])
    with col_mode:
        mode = st.radio(
            "Resource Type",
            ["Single Instance", "Multiple Instance"],
            horizontal=True
```

```
        )
    with col_p:
        num_processes = st.number_input(
            "Number of Processes (P)",
            min_value=2, max_value=10, value=3, step=1
        )
    with col_r:
        num_resources = st.number_input(
            "Number of Resources (R)",
            min_value=1, max_value=10, value=3, step=1
        )

st.markdown("<br>", unsafe_allow_html=True)


with st.container(border=True):
    st.subheader("1️⃣ Total Resource Capacity")
    st.caption("Maximum number of instances available for each resource type.")

    total_resources = []
    total_cols = st.columns(num_resources)
    for j, col in enumerate(total_cols):
        with col:
            val = st.number_input(
                f"R{j} (Total Capacity)",
                min_value=1,
                max_value=100,
                value=7 if j == 0 else 5,
                step=1,
```

```python
                step=1,
                key=f"total_r{j}"
            )
            total_resources.append(val)

total = np.array(total_resources, dtype=int)

st.markdown("<br>", unsafe_allow_html=True)


st.subheader("2 Current System State")

col_alloc, col_req = st.columns(2)


with col_alloc:
    st.info("Allocation Matrix (Resources currently held)")
    default_alloc = pd.DataFrame(
        0,
        index=[f"P{i}" for i in range(num_processes)],
        columns=[f"R{j}" for j in range(num_resources)]
    )
    alloc_df = st.data_editor(
        default_alloc,
        num_rows="fixed",
        use_container_width=True,
        key="alloc_editor"
    )

```


```python
                step=1,
                key=f"total_r{j}"
            )
            total_resources.append(val)

total = np.array(total_resources, dtype=int)

st.markdown("<br>", unsafe_allow_html=True)


st.subheader("2 Current System State")

col_alloc, col_req = st.columns(2)


with col_alloc:
    st.info("Allocation Matrix (Resources currently held)")
    default_alloc = pd.DataFrame(
        0,
        index=[f"P{i}" for i in range(num_processes)],
        columns=[f"R{j}" for j in range(num_resources)]
    )
    alloc_df = st.data_editor(
        default_alloc,
        num_rows="fixed",
        use_container_width=True,
        key="alloc_editor"
    )

```

```python
with col_req:
    st.warning("🟧 Request Matrix (Remaining resources needed to finish)")
    default_req = pd.DataFrame(
        0,
        index=[f"P{i}" for i in range(num_processes)],
        columns=[f"R{j}" for j in range(num_resources)]
    )
    req_df = st.data_editor(
        default_req,
        num_rows="fixed",
        use_container_width=True,
        key="req_editor"
    )

st.markdown("---")


st.subheader("3️⃣ Automatically Computed Available Resources")

alloc = alloc_df.to_numpy(dtype=int)
req = req_df.to_numpy(dtype=int)

allocated_sum = alloc.sum(axis=0)
available = total - allocated_sum


avail_cols = st.columns(num_resources)
for j, col in enumerate(avail_cols):
    with col:
        if available[j] < 0:
            col.metric(f"R{j}", int(available[j]), delta="Invalid (Negative!)")
        else:
            col.metric(f"R{j}", int(available[j]))

if np.any(available < 0):
    st.error("🚨 Allocated resources exceed Total Capacity! Please correct values.")
    st.stop()

st.markdown("---")


st.subheader("4️⃣ Simulation Analysis")

c1, c2, c3 = st.columns([1, 2, 1])
with c2:
    run_btn = st.button("🚀 DETECT DEADLOCK", type="primary")

def bankers_deadlock(total_vec, alloc_mat, req_mat):
    """
    total_vec : (m,) total capacity of each resource
    alloc_mat : (n,m) allocation
    req_mat   : (n,m) remaining need / request
    returns (is_deadlock, safe_sequence, deadlocked_processes, final_available)
    """
    total_vec = np.array(total_vec, dtype=int)
    alloc_mat = np.array(alloc_mat, dtype=int)
    req_mat = np.array(req_mat, dtype=int)

```

```python

    n, m = alloc_mat.shape
    work = total_vec - alloc_mat.sum(axis=0)
    finish = np.array([False] * n)
    safe_seq = []

    changed = True
    while changed:
        changed = False
        for i in range(n):
            if not finish[i] and np.all(req_mat[i] <= work):

                work = work + alloc_mat[i]
                finish[i] = True
                safe_seq.append(f"P{i}")
                changed = True

    deadlocked = [f"P{i}" for i in range(n) if not finish[i]]
    is_deadlock = len(deadlocked) > 0
    return is_deadlock, safe_seq, deadlocked, work

if run_btn:

    st.session_state["last_state"] = {
        "total": total.tolist(),
        "alloc": alloc.tolist(),
        "req": req.tolist(),
    }

        time.sleep(0.4)

        is_dead, safe_seq, deadlocked, final_avail = bankers_deadlock(total, alloc, req)

        status.update(label="Analysis Complete", state="complete", expanded=False)
    if not is_dead:
        label_mode = "Single Instance" if mode == "Single Instance" else "Multiple Instance"
        st.markdown(f"""
        <div class="result-box-safe">
            <h2>✅ SAFE STATE ({label_mode})</h2>
            <h4>Safe Sequence: {" → ".join(safe_seq) if safe_seq else "None (all zero requests)"}</h4>
        </div>
        """, unsafe_allow_html=True)

        st.markdown("<br>", unsafe_allow_html=True)
        st.caption("Final Available Resources after completing all processes:")
        cols_metric = st.columns(len(final_avail))
        for j, val in enumerate(final_avail):
            cols_metric[j].metric(f"R{j}", int(val), delta="Freed")
    else:
        st.markdown(f"""
        <div class="result-box-dead">
            <h2>💀 DEADLOCK DETECTED</h2>
            <p>Type: {"Single Instance" if mode == "Single Instance" else "Multiple Instance"}</p>
            <p>Blocked Processes: {", ".join(deadlocked)}</p>
        </div>
        """, unsafe_allow_html=True)

```

LOVELY PROFESSIONAL UNIVERSITY

LIVE PROJECT LINK :

https://deadlock-detection-and-recovery-toolkit-proj.streamlit.app/

THANK YOU